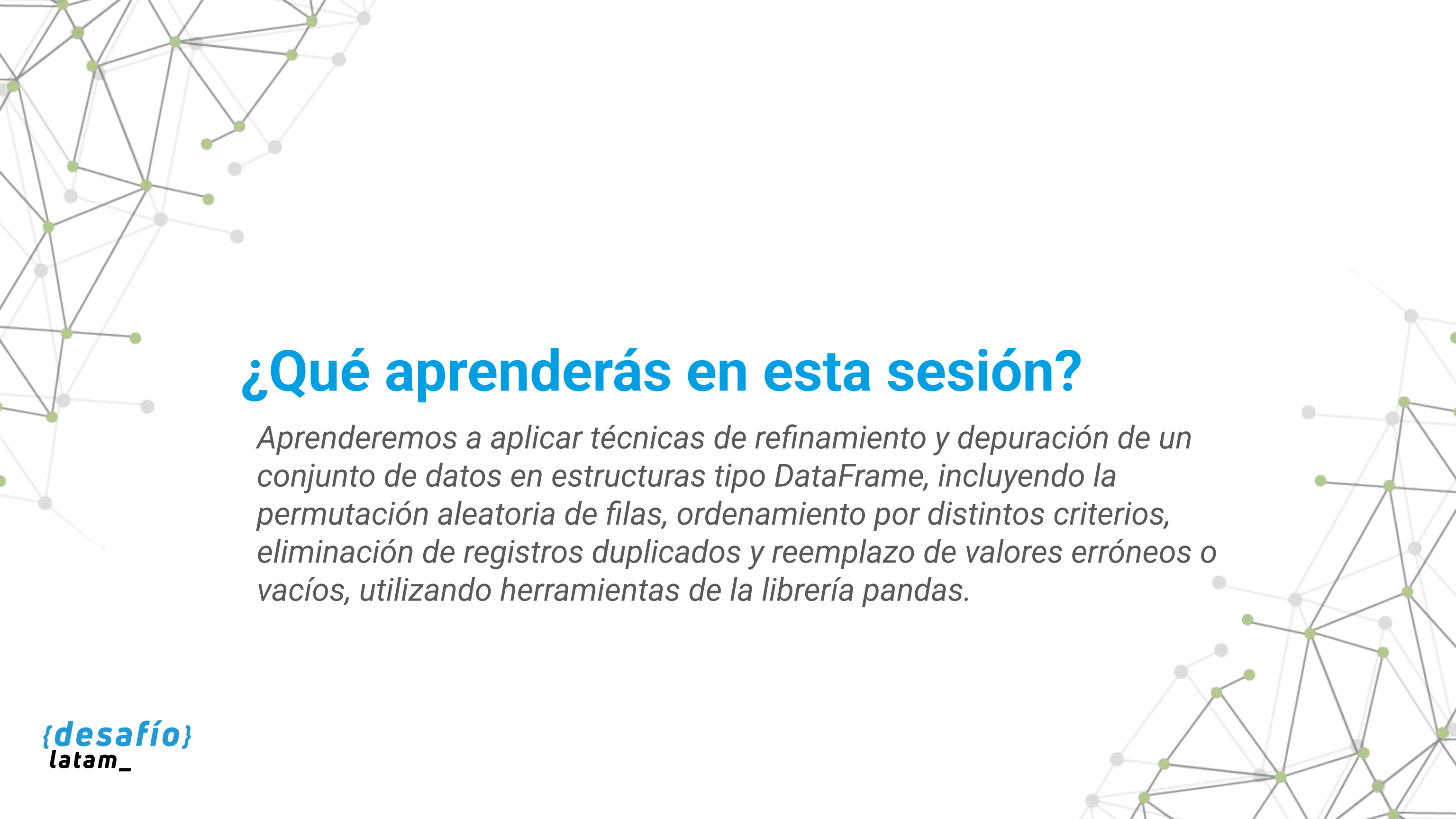




Limpieza y preparación de datos

Clase 4 - Refinamiento de datos y tratamiento de duplicados y valores erróneos



¿Qué aprenderás en esta sesión?

Aprenderemos a aplicar técnicas de refinamiento y depuración de un conjunto de datos en estructuras tipo DataFrame, incluyendo la permutación aleatoria de filas, ordenamiento por distintos criterios, eliminación de registros duplicados y reemplazo de valores erróneos o vacíos, utilizando herramientas de la librería pandas.

Desafío inicial

Trabajas como analista en el área de fidelización de clientes de una empresa de retail. Recibes una base de datos con registros de personas que han realizado compras, pero detectas lo siguiente:

- Algunos clientes aparecen más de una vez.
- Existen registros con el mismo nombre pero distinta región.
- Algunas filas están desordenadas sin seguir un criterio claro.
- Aparecen valores de contacto como None o inconsistencias como "altaa".

Tu objetivo es entregar un archivo limpio y depurado, listo para segmentar campañas.

¿Cómo aplicarías técnicas de depuración para asegurar que la base de datos sea confiable y usable?

/* Permutación de la data */

Permutación de la data

¿Qué es la permutación de datos y por qué es útil más allá del muestreo?

La permutación aleatoria en pandas se realiza cuando queremos reorganizar completamente el orden de las filas de un DataFrame, sin modificar su contenido. Esto es especialmente útil cuando el orden original de los datos podría generar sesgos en el análisis o entrenamiento de modelos.

Usos prácticos

- Simular condiciones de entrada aleatoria en modelos.
- Validar que los resultados no dependen del orden secuencial.
- Visualizar patrones ocultos al "barajar" los datos.

Recomendaciones

- Siempre usar `reset_index(drop=True)` para evitar que los índices antiguos persistan y generen confusión.
- Usar `random_state` para garantizar resultados repetibles (reproducibilidad).



Sintaxis de permutación

```
df.sample(frac=1).reset_index(drop=True)
```

- frac=1: toma el 100% de las filas y las reordena.
- reset_index(drop=True): reindexa el DataFrame y elimina el índice anterior.

Ejemplo aplicado:

```
clientes_permutado = df.sample(frac=1,  
random_state=42).reset_index(drop=True)
```

Esto asegura que la permutación sea reproducible (útil en producción o pruebas).

Ahora que sabemos cómo reorganizar aleatoriamente nuestros datos, pasemos a una técnica más estructurada: el ordenamiento según criterios definidos, clave para interpretar información secuencialmente o generar reportes jerárquicos.



Ordenamiento de DataFrames

1

¿Cómo ordenar filas según una o más columnas?

El ordenamiento de un DataFrame se realiza con la función `sort_values()`.

Puedes ordenar por:

- Una columna (ej. `compras`)
- Múltiples columnas (ej. `region` y `nombre`)

2

Claves del ordenamiento

- `ascending=True/False` define la dirección del orden.
- Puedes ordenar alfabéticamente, numéricamente o por fechas.
- También se puede ordenar por índices (cuando estos tienen significado jerárquico o temporal).

3

Ejemplos de código

```
df.sort_values(by='compras',  
ascending=False)
```

Para ordenar por múltiples columnas:

```
df.sort_values(by=['region',  
'nombre'], ascending=[True, False])
```

Buenas prácticas de ordenamiento

Buenas prácticas

- Usar `inplace=True` solo si se quiere modificar el DataFrame original.
- Comprobar siempre que los tipos de datos son los adecuados para el ordenamiento (ej. no strings en campos numéricos).

Hasta aquí hemos reorganizado datos por criterios aleatorios o definidos. Pero ¿qué pasa si hay filas duplicadas que podrían afectar los resultados de un análisis? Veamos cómo detectarlas y eliminarlas con precisión.



/* Eliminación de registros duplicados */

Eliminación de registros duplicados

¿Qué tipos de duplicados existen?

Hay dos situaciones comunes:

- Duplicados completos: filas que coinciden en todas sus columnas.
- Duplicados parciales: filas con un valor clave repetido pero con variaciones (por ejemplo, mismo cliente_id pero distinta región).

¿Cómo identificar y eliminar filas repetidas?

Un duplicado es una fila idéntica a otra (en todas o algunas columnas). Para tratarlos, usamos:



Detección y eliminación de duplicados

Detección

```
df.duplicated()
```

Esto retorna una serie booleana con True donde hay duplicados (no en la primera ocurrencia).

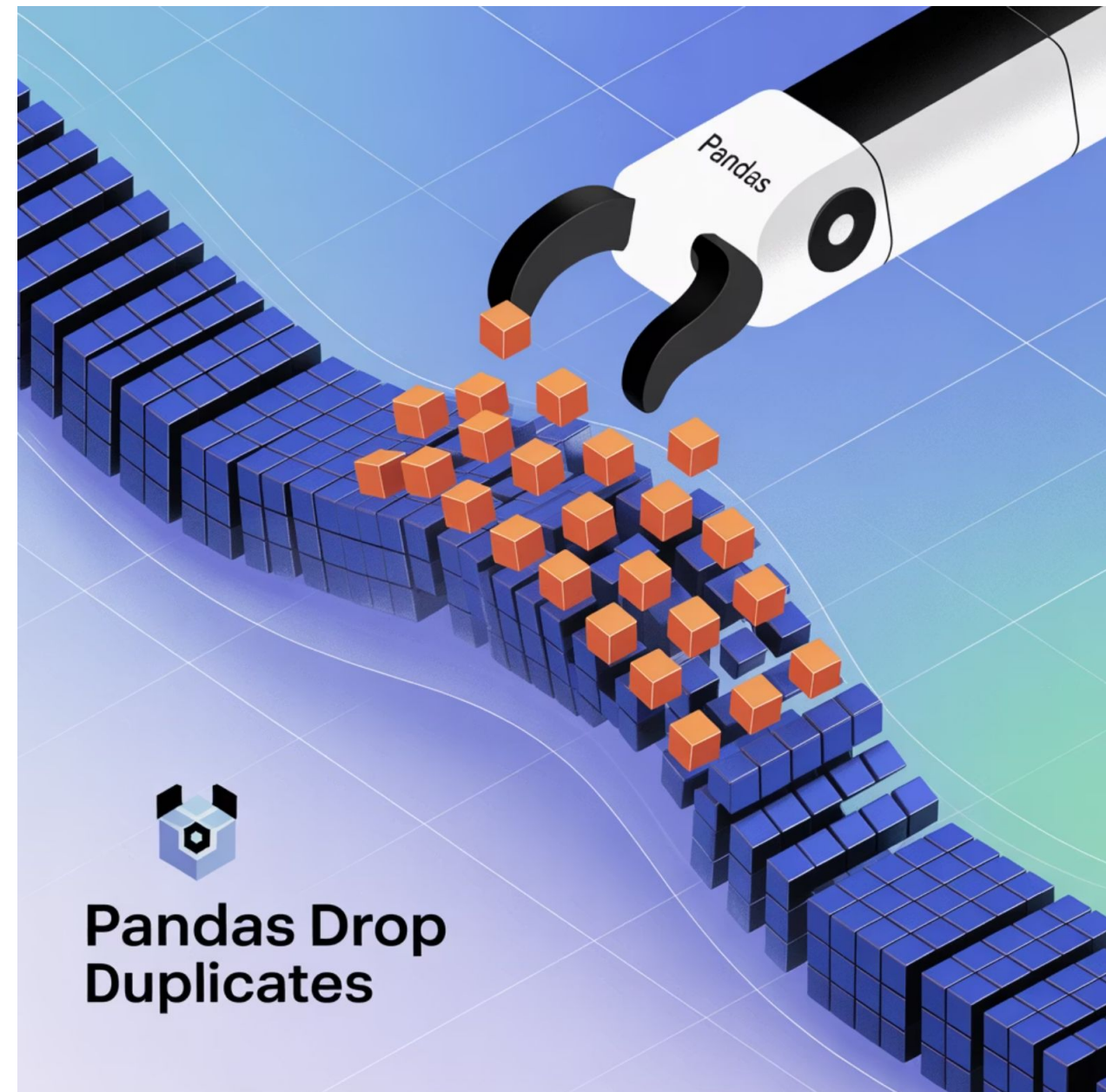
Eliminación

```
df_sin_duplicados = df.drop_duplicates()
```

Por columnas específicas

```
df.drop_duplicates(subset=['cliente_id'])
```

Esto elimina entradas donde el cliente_id se repite, manteniendo la primera.



Consideraciones sobre duplicados

Consideraciones

- Se puede usar el argumento `keep='first'` o `keep='last'` para decidir cuál fila conservar.
- Eliminar duplicados no siempre es la mejor solución: a veces es necesario revisar y combinar información.

Con los duplicados controlados, pasamos al último paso esencial de esta sesión: reemplazar valores incompletos o incorrectos para estandarizar la base de datos.



/* Reemplazo de valores */

Reemplazo de valores

¿Cuándo reemplazar y cuándo eliminar?

El reemplazo de valores es útil para corregir errores de digitación, homogeneizar nomenclaturas o manejar datos faltantes sin perder registros útiles.

Tipos de reemplazo

- Un valor por otro: errores comunes ("altaa" → "alta").
- Múltiples reemplazos simultáneos: útil para categorías o etiquetas.
- Valores nulos: con `fillna()` o directamente con `replace()`.

Opciones de reemplazo

¿Qué opciones existen para reemplazar valores erróneos o vacíos y dejar los datos listos para análisis?
pandas nos ofrece funciones potentes para buscar y reemplazar.

Métodos de reemplazo en pandas

Reemplazar valores específicos

```
df['frecuencia_contacto'].replace('altaa', 'alta',  
inplace=True)
```

Reemplazar múltiples valores

```
df.replace({'altaa': 'alta', None: 'desconocido'},  
inplace=True)
```

Reemplazar con condiciones

```
df.loc[df['compras'] == 0, 'compras'] = 1
```

Esto puede usarse para corregir datos inválidos sin necesidad de eliminarlos.



Buenas prácticas de reemplazo

Validar valores únicos

Siempre verificar los valores únicos antes de realizar cualquier reemplazo para entender el alcance de los cambios.

Documentar decisiones

Documentar las decisiones de reemplazo como parte del flujo de transformación para mantener la trazabilidad.

Preservar datos originales

Evitar sobrescribir sin copia previa del dataset (`df.copy()`) para mantener la integridad de los datos originales.



Actividad guiada: Limpieza final de registros de clientes



Ejercicio

Limpieza final de registros de clientes

Objetivo de la actividad

Aplicar técnicas de permutación, ordenamiento, detección de duplicados y reemplazo de valores para depurar un set de datos realista de clientes.

Situación problema

El equipo de CRM solicita una base de datos depurada para análisis de fidelización. Tú debes entregarla limpia, sin duplicados, con valores corregidos y en el orden requerido.



Ejercicio

Instrucciones

Cargar el archivo

```
import pandas as pd
df =
pd.read_csv('clientes_registros.csv')
)
```

Permutar y ordenar

1. Permutar el orden de las filas.
2. Ordenar por 'compras' de mayor a menor.

Eliminar duplicados y corregir valores

3. Eliminar duplicados por cliente_id.
4. Reemplazar valores nulos en 'frecuencia_contacto' con 'desconocido'.
5. Corregir 'altaa' por 'alta'.
6. Guardar el resultado como clientes_limpios.csv.



Data Quality Impact on Business Decisions

Preguntas de reflexión y cierre

Criterio para duplicados

¿Qué criterio usaste para decidir qué duplicado mantener?

Impacto en campañas

¿Qué impacto tienen los errores de ortografía o valores vacíos en una campaña comercial?

Diferencia conceptual

¿Qué diferencia hay entre eliminar un duplicado y reemplazar su valor?

Actividad práctica autónoma: Depuración de datos de productos defectuosos



Ejercicio

Depuración de datos de productos defectuosos

Objetivo de la actividad

Aplicar técnicas de depuración en un set de datos sobre reclamos de productos, detectando duplicados y corrigiendo etiquetas inconsistentes.

Contexto

Trabajas con un archivo de reclamos que contiene datos duplicados y desordenados. Algunos registros tienen la categoría de defecto mal escrita, como "electrónico" o "sin categ."



Ejercicio

Instrucciones



Crear DataFrame

```
import pandas as pd
df = pd.DataFrame({
    'reclamo_id': [11, 12, 13,
14, 15, 13],
    'producto': ['TV', 'Celular',
'Tablet', 'TV', 'Celular',
'Tablet'],
    'categoria_defecto':
['electrónico', 'electrónico',
'electrónico', 'electrónico', 'sin
categ.', 'electrónico']
})
```



Depurar datos

1. Eliminar duplicados por reclamo_id.
2. Reemplazar 'electrónico' → 'electrónico'.
3. Reemplazar 'sin categ.' → 'desconocido'.



Finalizar

4. Ordenar por 'producto'.
5. Guardar como reclamos_limpios.csv.

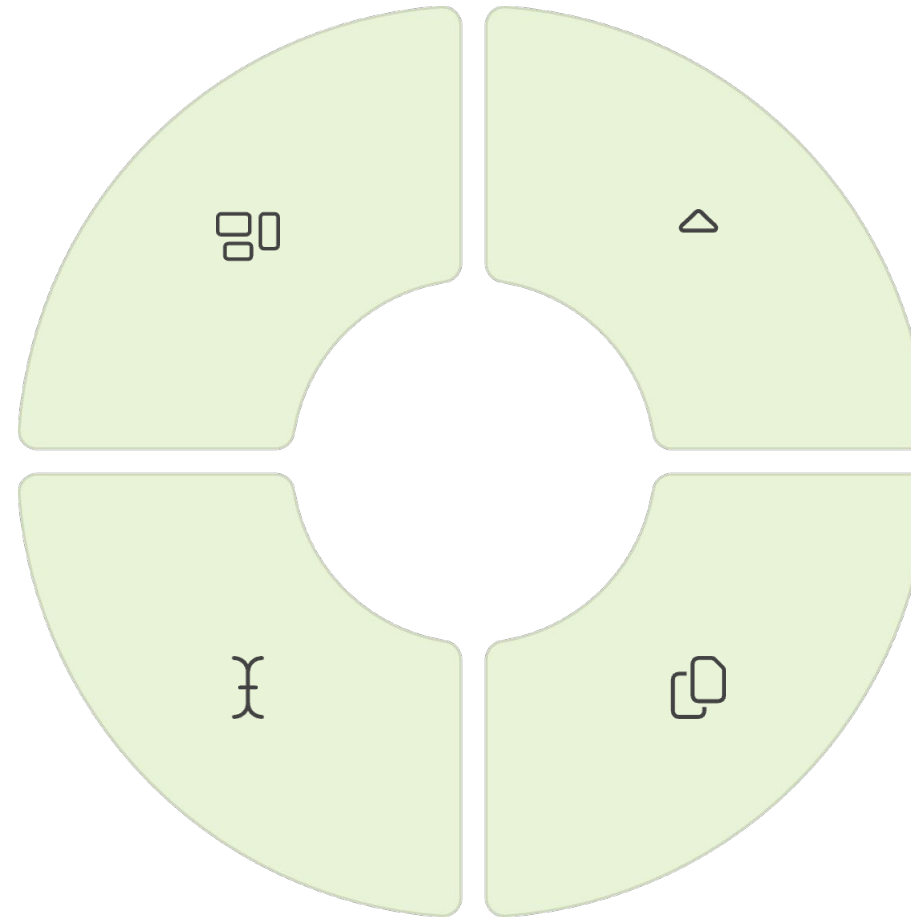
Resumen

Permutación

Aprendimos a permutar datos aleatoriamente para romper sesgos de orden.

Reemplazo

Reemplazamos valores erróneos, nulos o inconsistentes.

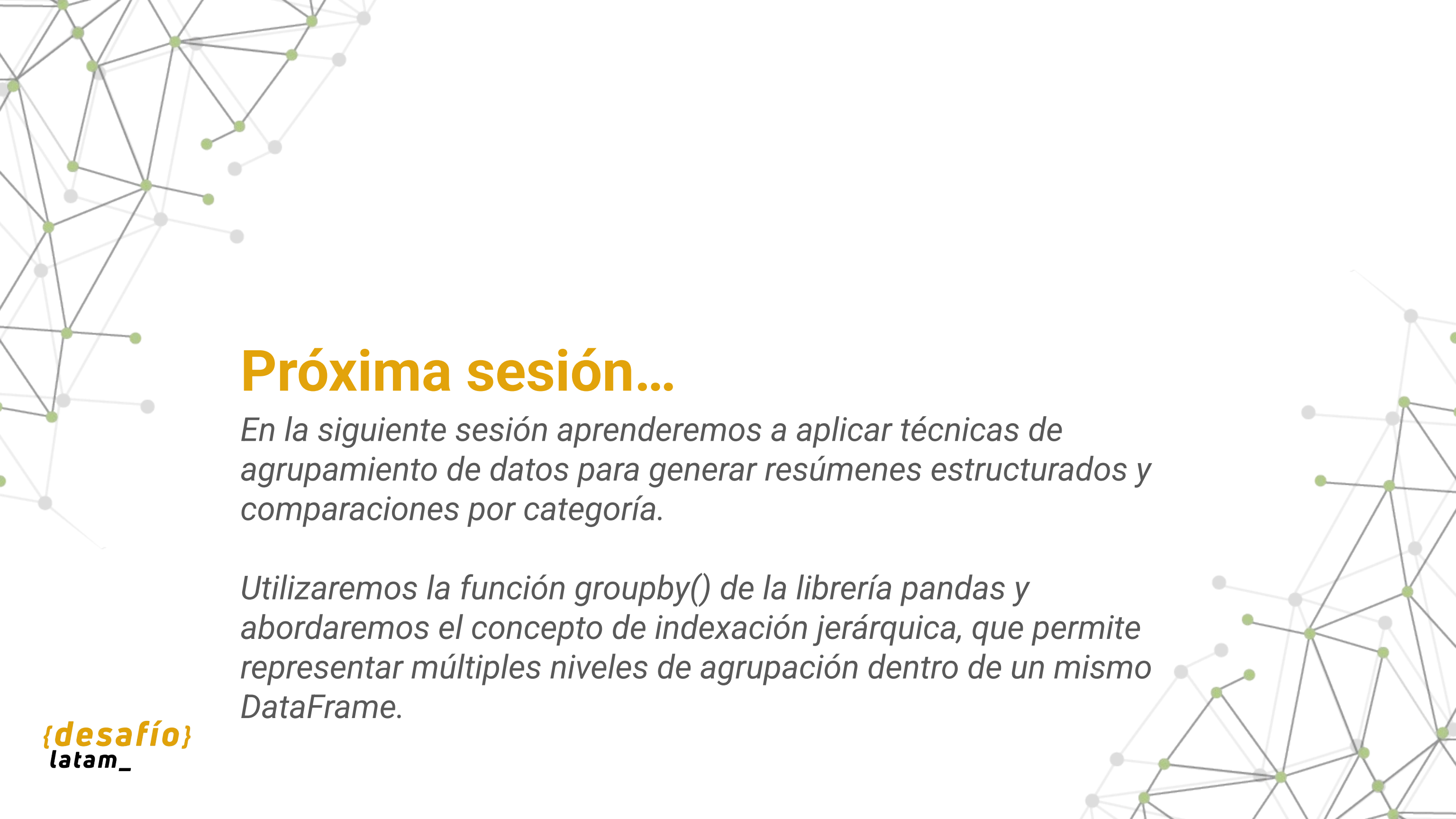


Ordenamiento

Aplicamos ordenamiento por una o varias columnas.

Duplicados

Detectamos y eliminamos duplicados para evitar sesgo analítico.



Próxima sesión...

En la siguiente sesión aprenderemos a aplicar técnicas de agrupamiento de datos para generar resúmenes estructurados y comparaciones por categoría.

Utilizaremos la función `groupby()` de la librería pandas y abordaremos el concepto de indexación jerárquica, que permite representar múltiples niveles de agrupación dentro de un mismo DataFrame.